

PROJECT REPORT

Hiring Intelligence Platform

A React + Vite single-page application that simulates an AI-assisted recruitment workflow — job postings, resume parsing, candidate ranking, a natural-language hiring assistant, and hiring analytics — built with a mock-first API layer that mirrors a real backend contract.

Frontend Codebase Walkthrough & Build Pipeline Explanation
Prepared for academic project submission

1. What This Project Is

The **Hiring Intelligence Platform** is a web application that helps recruiters manage the hiring pipeline: posting jobs, uploading candidate resumes, automatically "parsing" them into structured data, ranking candidates against a job's requirements with a match score, chatting with an AI assistant to search candidates in plain English, and viewing analytics dashboards on hiring funnels and skill gaps. It also has a separate, simpler portal for candidates to upload their own resume and check their application status.

It is a **frontend-only** project at this stage: there is no real backend yet. Every API call is routed through a mock layer that returns realistic fake data with simulated network delay, so the entire user experience — loading states, error states, forms, tables, charts — works and can be demoed end-to-end, even though nothing is persisted on a real server.

2. Technology Stack

Layer	Technology	Why it's used
UI Library	React 19	Component-based UI; the app is built entirely from reusable function components and hooks.
Build Tool	Vite 8	Fast dev server with Hot Module Reload (HMR) and an optimized production bundler.
Routing	react-router-dom 7	Client-side routing with nested layouts and role-based route guards.
Styling	Tailwind CSS 4	Utility-first CSS, integrated directly into Vite via the official <code>@tailwindcss/vite</code> plugin (no separate PostCSS config needed).
Forms	react-hook-form	Performant form state management/validation for the Job creation/edit form.
Charts	recharts	Renders the Analytics dashboard's bar/line/funnel charts.
HTTP Client	axios	Wrapped in a shared client with auth-token injection, ready for a real backend.
Linting	oxlint	Fast Rust-based linter for catching code issues.

3. The Build & Run Pipeline

The project follows the standard modern frontend pipeline: **write** → **dev-serve** → **lint** → **build** → **preview/deploy**.

npm install

→ installs all dependencies from package.json / package-lock.json

npm run dev

→ starts Vite's dev server (http://localhost:5173) with instant HMR

npm run lint

→ runs oxlint across the codebase for style/error checking

npm run build

→ Vite bundles + minifies the app into static files in /dist

npm run preview

→ serves the /dist production build locally to sanity-check it

Under the hood, `vite.config.js` registers two plugins: `@vitejs/plugin-react` (compiles JSX, enables Fast Refresh) and `@tailwindcss/vite` (scans class names and generates only the CSS that is actually used). `src/main.jsx` is the true entry point — it mounts the `<App />` component into the `#root` div declared in `index.html`, wrapped in React's `StrictMode` for extra development-time checks.

4. High-Level Architecture

The codebase is organised by responsibility, which is a common pattern in production React apps:

Folder	Responsibility
src/pages/	One component per route/screen (Dashboard, Jobs, Resume Upload, etc.) — composes smaller components together.
src/components/	Reusable UI pieces, grouped by feature area: <code>common/</code> (Button, Card, Modal, Loader), <code>layout/</code> (Sidebar, Topbar, route guards), <code>jobs/</code> , <code>candidates/</code> , <code>resume/</code> , <code>analytics/</code> , <code>chat/</code> .
src/api/	One file per domain (jobsApi, resumeApi, candidatesApi, analyticsApi, reportsApi, assistantApi, authApi, dashboardApi) — the only layer allowed to talk to the network.
src/api/mockData/	Static fake datasets (users, jobs, candidates, analytics, reports) used while there is no real backend.
src/context/	React Context providers — currently <code>AuthContext</code> , which holds the logged-in user and auth actions globally.
src/utils/	Small pure helper functions, e.g. <code>redirectByRole</code> .
src/App.jsx	Defines every route and which layout/guard wraps it.

Design principle: Pages never call `axios` directly. They import functions from `src/api/*`. This means the day a real backend exists, only the `api/` layer needs to change — no page or component has to be touched.

5. Authentication & Role-Based Routing

Authentication state lives in `AuthContext.jsx` using React's Context + `useState`. On login or register, the API layer returns a token plus user info; both are saved to `localStorage` (`hip_token`, `hip_user`) so the session survives a page refresh, and the user object is kept in memory via `useContext` so any component can call `useAuth()` to read it.

Route protection

`App.jsx` builds the router with `createBrowserRouter`. Two role groups are fenced off by a shared `ProtectedRoute` component:

Recruiter / Admin routes

Dashboard, Jobs, Resume Upload, AI Assistant, Analytics, Reports — guarded by `allowedRoles=['RECRUITER', 'ADMIN']`.

Candidate routes

Upload Resume, Application Status — guarded by `allowedRoles=['CANDIDATE']`.

`ProtectedRoute` checks `isAuthenticated` from `useAuth()`: if false it redirects to `/login` (remembering where the user was headed); if the user's role isn't in the allowed list it redirects to `/unauthorized`; otherwise it renders the nested route via React Router's `<Outlet />`. `RootRedirect` and `redirectByRole.js` send a freshly logged-in user to the right home screen depending on whether their role is `CANDIDATE` or not.

6. The Mock API Layer (the heart of the "pipeline")

Every domain file in `src/api/` follows the same shape, controlled by a single flag in `mockHelpers.js`:

```
export const USE MOCK_API = true;

export function mockDelay(data, ms = 500) {
  return new Promise((resolve) => setTimeout(() => resolve(data), ms));
}
```

Each exported API function branches on that flag:

```
export async function getJobs() {
  if (USE MOCK_API) {
    return mockDelay([... jobsStore]); // fake data, fake latency
  }
  const { data } = await axiosClient.get('/jobs'); // real backend call
  return data;
}
```

This is why the entire demo feels realistic: `mockDelay` simulates network latency so loading spinners actually show up, and `mockError` simulates failures. `axiosClient.js` (built on `axios`) is fully wired for the real backend already — it reads `VITE_API_BASE_URL` from environment variables, automatically attaches the JWT bearer token from `localStorage` to every request, and clears the session on a 401 response. **Flipping `USE MOCK_API` to `false` is the entire migration step to a live backend** — no component code changes.

Resume "parsing" is a good example of how convincingly the mock layer imitates a real async job:

```
uploadResumes(files)
└─ for each file:
    onFileStatusChange(file, 'uploading') ┌
    ... simulated delay ...                │ UI updates live via
    onFileStatusChange(file, 'parsing')   └─ a callback, exactly like
    ... simulated delay ...                │ it would with real
    onFileStatusChange(file, 'done'|'error') └─ upload+parse progress events
```

7. Feature Walkthrough

7.1 Dashboard

`DashboardPage.jsx` fetches a summary object (active job count, resumes processed, average match score, recent jobs, recent activity feed) from `dashboardApi.js` in a `useEffect` on mount, and renders it as stat cards plus two list panels, all built from the shared `Card` component.

7.2 Job Management

Recruiters can list jobs (`JobsListPage` / `JobTable`), create or edit a job (`JobFormPage` using `react-hook-form` for validation), and open a job's detail page. `jobsApi.js` keeps an in-memory `jobsStore` array that `createJob` / `updateJob` mutate, so changes persist for the session even though there is no database yet.

7.3 Resume Upload & Parsing

`UploadDropzone.jsx` implements drag-and-drop plus click-to-browse file selection, filtering to PDFs only. Selected files flow into `ResumeUploadPage`, which tracks a status per file (`uploading` → `parsing` → `done/error`) shown live in `UploadProgressList`,

and finally renders extracted fields (name, skills, experience, education) in `ParsedDataTable` .

7.4 Candidate Ranking

`JobDetailPage` calls `getJobCandidates(jobId)` , which cross-references every candidate's applications against that job, computes a `matchScore` , and returns candidates sorted best-first. Each row shows a colour-coded `MatchScoreBadge` , skill tags, strengths and missing skills — this is the core "hiring intelligence" feature: turning raw resumes into a ranked shortlist.

7.5 AI Chat Assistant

`AssistantPage` renders a chat UI (`ChatWindow` , `ChatMessage` , `ChatInput`). When the recruiter types a natural-language query like "Find candidates with Java and AWS with 3+ years experience", `assistantApi.js` extracts known skill keywords and a minimum-experience number straight out of the query text with simple string matching/regex, filters the candidate pool accordingly, and returns a natural-language answer plus the matching candidate list — a lightweight stand-in for what a real LLM-backed endpoint would do server-side.

7.6 Analytics

`AnalyticsPage` fetches four independent datasets in parallel (`Promise.all`) — skill analytics, hiring funnel, skill gap, monthly trends — and renders each with a dedicated `recharts` -based component sharing one visual theme (`chartTheme.js`).

7.7 Reports & Candidate Portal

`ReportsPage` surfaces exportable hiring reports. Separately, a lightweight `CANDIDATE` -role portal (`CandidateResumeUploadPage` , `ApplicationStatusPage`) lets an applicant upload their own resume and track status, reusing the same upload components.

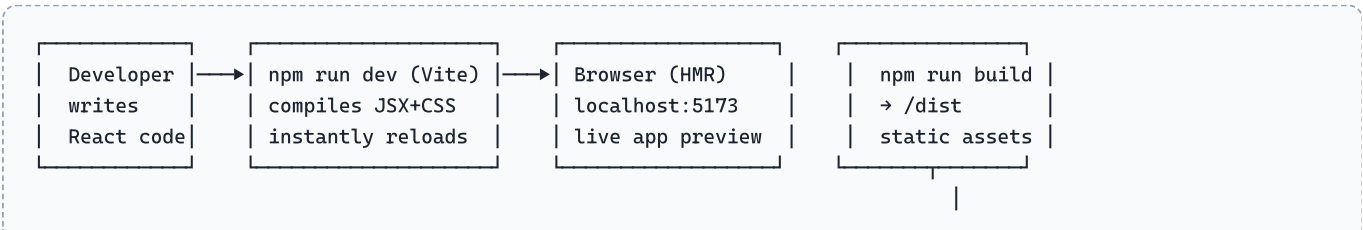
8. Shared UI Component Library

`src/components/common/` holds small, prop-driven building blocks used everywhere — `Button` (variant/size props), `Card` (title/action slot/body), `Modal` , `Loader` (spinner + label), `EmptyState` . Keeping these generic and presentation-only (no data fetching) is what lets every page look consistent without copy-pasting markup.

9. Styling Approach

Styling is done entirely with Tailwind CSS utility classes directly in JSX (e.g. `"flex items-center gap-3 rounded-md px-3 py-2 text-sm font-medium"`) — there are no separate `.css` files per component. Tailwind v4 is wired in via its Vite plugin, so `src/index.css` only needs a single `@import "tailwindcss";` line; Tailwind scans the source files at build time and generates just the CSS actually referenced, keeping the shipped stylesheet small.

10. How to Explain the "Pipeline" in One Diagram



▼

Hosting (Netlify, Vercel, S3, etc.)

Runtime data flow (current, mock-backed):

Page component → api/*.js function → USE MOCK_API ? mock data+delay : axiosClient → (future) real backend

11. Path to a Real Backend

1. Stand up real REST endpoints matching the shapes already assumed by `api/*.js` (e.g. `GET /jobs`, `POST /resumes/upload`, `POST /assistant/query`).
2. Set `VITE_API_BASE_URL` in a `.env` file to point at that backend.
3. Flip `USE MOCK_API` from `true` to `false` in `mockHelpers.js`.
4. No page or component code needs to change — the API layer is the only seam.

12. Summary Talking Points for Presentation

- **Frontend framework:** React 19 with functional components and hooks only (no class components).
- **Build tool:** Vite for near-instant dev feedback and an optimized production bundle.
- **Architecture:** Clear separation of pages, reusable components, API layer, and context — a scalable pattern used in real production codebases.
- **Security-minded routing:** role-based access control via a single reusable `ProtectedRoute` guard.
- **Backend-ready design:** a mock API layer that mirrors the real contract, so swapping in a live backend is a one-line change, not a rewrite.
- **Feature completeness for a demo:** auth, job CRUD, resume upload/parsing simulation, AI-style natural-language candidate search, analytics dashboards, and a separate candidate-facing portal.